Code Editor Brief: IncuBrix Lead Engine

Hand this file to the code editor (Claude Code, Cursor, Copilot or similar) at the start of the project. It defines what to build, how to build it, and the documentation you must keep current.

1. Project in one paragraph

Build a small Python engine that finds content creators (podcasters, YouTubers, expert-led creators), verifies each with public evidence, picks the best contact channel, drafts personalised outreach, and exports a CRM-ready `leads.csv`. It must be rerunnable weekly with one command. Target: 1,000+ unique qualified leads, 0% duplicates, 25+ Priority A leads with full sequences.

2. Working rules for the code editor

1. **Keep it simple.** Prefer plain functions, pandas and SQLite. No frameworks, no classes unless they remove real complexity. If a simpler version works, use it.
2. **Maintain documentation as you build (mandatory).** Every code change that alters behaviour, tools, config or workflow must update the matching doc in the same change. See section 3.
3. **One stage, one file.** Each pipeline stage reads a table and writes a table, so it can be run alone.
4. **Never commit secrets.** Keys live in `.env`; commit only `.env.example`.
5. **No invented data.** Never fabricate leads, contacts, handles, evidence or metrics. If a source fails, log it and skip the row.
6. **No real outreach.** The engine may only send to test accounts the owner controls.
7. **Test as you go.** Add or update a test for each rule you implement. Run tests before marking a phase done.
8. **Explainable code.** The owner must explain every line in a live review, so add short comments on the why, not the what.
9. **Ask before adding a dependency**, and record it in the tech docs when added.

3. Documentation to maintain

Create these in Phase 0 and keep them accurate at every step.

File	What it holds	Update when
`README.md`	what the project is, setup, the one run command, config, outputs, troubleshooting	any setup, command or output changes
`docs/TECH_BUILD.md`	tools used to **build** the app: language version, editor and AI tools, git, test and lint tools, environment setup	a dev tool is added, removed or upgraded
`docs/TECH_RUNTIME.md`	tech used **inside** the app: libraries, APIs, data stores, each with its purpose, quota or cost, and the file that uses it	a library, API or data store changes
`docs/WORKFLOW.md`	the app workflow: stages, inputs, outputs, failure handling	a stage or its rules change
`docs/PHASES.md`	phase list with status (not started, in progress, done) and what was finished	at the end of each phase
`docs/DECISIONS.md`	short log: decision, reason, date	any non-obvious choice, threshold or trade-off
`CHANGELOG.md`	dated list of notable changes	every meaningful commit
`output/run_report.md`	generated each run: counts, mix, runtime, API calls, cost, errors	generated by the engine, do not hand-edit

Rule: if code and docs disagree, fix the docs in the same change. A phase is not done until its docs are updated.

4. Tech used to build the app

Area	Tool
Language	Python 3.11+
Environment	`venv` with a pinned `requirements.txt`
Version control	Git, private GitHub repository
Editor and AI assist	the code editor in use, plus any AI tools (list every one in `docs/TECH_BUILD.md` and in the workbook)
Testing	`pytest`
Lint and format	`ruff`
Config and secrets	`.env` (local, ignored by git), `.env.example` (committed), `config.yaml`

5. Tech used inside the app

Purpose	Library or service	Notes
Data handling	`pandas`	all stage tables
Storage and cache	SQLite (`sqlite3`)	candidates, API cache, run state
HTTP and retries	`requests` , `tenacity`	3 attempts with backoff
YouTube discovery	YouTube Data API v3	`search.list` costs 100 quota units per call, so use sparingly; prefer `channels.list` and `playlistItems.list`
Podcast discovery	Podcast Index API, iTunes Search API, RSS via `feedparser`	free tiers
Website reading	`requests` plus `beautifulsoup4`	public pages only, respect `robots.txt` and site terms
Language detection	`langdetect`	English content check

Purpose	Library or service	Notes
Domain normalisation	`tldextract`	dedupe key
Classification and drafting	one LLM API with JSON output	may only quote from supplied text; must return an evidence quote and URL
Config	`PyYAML`, `python-dotenv`	thresholds, weights, caps
Email seed test	`smtplib`, `imaplib`	test accounts only
Logging	Python `logging`	one log file per run

6. App workflow

```
seeds.csv
   |
   v
discover  -> candidate creators (platform ID, URL, source, query)
   |
   v
enrich    -> profile, recent uploads, links, public contact info (cached)
   |
   v
verify    -> 7 checks, each with evidence quote + URL; failures go to
rejected.csv
   |
   v
dedupe    -> one row per creator (platform ID, domain, email)
   |
   v
caps      -> country and segment rules enforced
   |
   v
score     -> 0 to 100, top 25 flagged Priority A
   |
   v
route     -> best channel + backup channel
   |
   v
```

```
draft     -> first message (+ 2 follow-ups for Priority A), opt-out included
   |
   v
export    -> leads.csv, outreach/, rejected.csv, run_report.md
```

Run command: `python run.py --input data/seeds.csv --out output/`

Refresh: `python run.py --refresh` re-checks evidence older than 30 days and keeps unchanged cached results.

Qualification checks (all must pass, each with a working public link):

1. Real and relevant: real creator, approved segment and country, official profile, stable platform ID.
2. Active: 8 or more items in 30 days, or 2 or more long-form items in 60 days.
3. Commercial: public sponsorship, product, service, course, membership or affiliate evidence.
4. Needs IncuBrix: public sign of a content creation, editing, captions, repurposing, publishing, consistency, backlog or team workflow need.
5. Reachable: real public business channel, active in the last 60 days, never guessed.
6. Personalised and compliant: cites a specific recent piece of content, names the sender, includes an opt-out.
7. Complete and unique: all fields filled, evidence checked within 30 days, no duplicates.

Hard constraints enforced in code: at least 80% from US, CA, UK, IE, AU, NZ, SG; at least 3 countries; no country above 60%; mainly English content. The run fails loudly if the final set breaks a rule.

Failure handling: retry 3 times, then log the error and skip the row. Stages write to SQLite, so a crashed run resumes from the last finished stage.

7. Phases

Finish each phase fully, update docs and tests, then move to the next.

Phase 0: Setup

- Create the repo layout, `venv`, `requirements.txt`, `.env.example`, `config.yaml`, `.gitignore`.
- Create all documentation files from section 3 with initial content.

- Confirm API access for YouTube, Podcast Index and the LLM.
- Lock the `leads.csv` columns to the Submission Workbook.
- **Done when:** `python run.py --help` works and docs exist.

Phase 1: Discover and enrich

- Implement discovery for YouTube and podcasts, then enrichment with caching.
- Run on a small seed set and inspect results by hand.
- **Done when:** 200 candidates are enriched with no repeat API calls on rerun. Update `TECH_RUNTIME.md` and `WORKFLOW.md`.

Phase 2: Verify, dedupe, caps

- Implement the 7 checks, evidence capture, dedupe and country/segment caps.
- Write `rejected.csv` with reasons.
- **Done when:** tests pass for the activity rule, dedupe, caps and missing evidence, and a manual check of 20 rows looks right.

Phase 3: Score and route

- Implement weighted scoring from `config.yaml`, Priority A flagging and channel routing.
- **Done when:** every lead has a best channel, a backup channel and a score, and weights are documented in `DECISIONS.md`.

Phase 4: Outreach drafting

- Write templates for 3 segments across 3 channels.
- Generate personalised first messages, plus 2 follow-ups for Priority A.
- Add automatic checks for content reference, sender line, opt-out and length.
- **Done when:** every Priority A lead has a complete sequence that passes the checks.

Phase 5: Scale, export, refresh

- Scale to 1,000 or more leads and implement the final export and `run_report.md`.
- Implement `--refresh` and resume after failure.
- Record runtime, API calls, tokens and cost for 1,000 rows (and 5,000 if attempted).
- **Done when:** a rerun produces 0% duplicates and an accurate run report.

Phase 6: Seed test

- Send the sequences from the engine to test accounts the owner controls on 3 channels.
- Prove scheduling, tracked links and reply logging, and save the proof.
- **Done when:** logs and screenshots exist for all 3 channels. Document any manual steps.

Phase 7: Hardening and handover

- Run a manual audit of 40 or more random leads and fix any weak rule.
- Re-check all evidence so it is dated within 30 days.
- Final pass on the README so a new person can run it from scratch.
- Final review of every doc against the code.
- **Done when:** the Definition of done below is fully checked.

8. Definition of done

- [ ] 1,000 or more unique qualified leads, 0% duplicates
- [ ] 25 Priority A leads with first message plus 2 follow-ups
- [ ] Country, segment and language rules pass
- [ ] Manual audit of 40 or more random leads at 90% or higher
- [ ] Seed test passed on 3 channels with proof
- [ ] One-command run and refresh both work from the README alone
- [ ] All documentation in section 3 is current and matches the code
- [ ] No secrets in the repository

9. Instruction to the code editor

Before you write any code, read this whole brief. Work one phase at a time. At the end of every phase, summarise what changed, list the docs you updated, and wait for confirmation before starting the next phase.